Quality Gates for Cooperation-Mode Labels: Detecting When a Behavioral Log Cannot Support Supervised Learning

Kazutaka Hayashi *kazutaka.hayashi@yorosix.com*
YOROSIX, Osaka, Japan

Abstract

Platforms that mediate cooperation (mutual-aid networks, referral systems, community request boards) log the raw material for supervised models of *how* people cooperate: pass by, broker a request onward, originate a sharing chain, or help directly. These operational logs are also the data most likely to contain defects that make such labels invalid while leaving predictive performance intact: administrative accounts mixed with genuine users, features recorded after the outcome, outcomes driven by sheer activity level, and too few positives. We define four cooperation-mode labels over four common log tables and a four-gate certification procedure (exclusion, minimum support, leakage, and an activity-baseline test) that a label must pass before supervised use, reported through a fixed-schema data card. Because no cooperative-platform dataset comes with ground truth about its own defects, we evaluate the gates by defect injection into a synthetic log with a known generative process. Over 500 replications per condition, a rule that certifies whenever cross-validated AUC exceeds 0.70 accepts data carrying administrative contamination, test accounts, leakage or activity confounding in 97.6%–100.0% of cases, and a stronger practitioner baseline still accepts all but the leakage case at 97.6%–100.0%; the gates accept the same data in 0.0%–3.0% of cases while certifying clean data in 100.0%. Ablations show each gate is necessary for at least one defect. We also report where the gates fail: leakage that is not declared in feature metadata and is weaker than the univariate tripwire passes undetected. Applied to eight months of logs from an operating mutual-aid platform, the gates decline to certify all four labels and separate a sample-size shortfall from a missing log field that no amount of further traffic would fix.

1 Introduction

A growing family of platforms mediates cooperation directly. A participant posts a request; others may respond, pass it on, or ignore it; and the platform logs every one of these events in ordinary operation. This invites training supervised models of cooperative behavior on data the platform already holds, with no new instrumentation or hand-labelling. The risk is structural. A log built to run a service, not to support inference, accumulates exactly the defects that make a supervised label invalid: operator and test traffic that looks like user behavior, features recorded after the outcome they are supposed to predict, outcomes explained by how active a participant is rather than by what they did, and outcomes too rare to estimate anything about. Each of these can leave cross-validated predictive performance looking healthy, which is why a performance threshold is not a safe certification rule.

We study this problem for one label family: *cooperation modes*. The outcome is not whether a participant cooperated but how: did they pass by, broker the request onward, originate a further chain of sharing, or help directly. The distinction matters for any system that routes requests or calibrates an agent-based model of how help moves through a population, because these roles call for different interventions.

Our contributions are as follows.

- **Operational labels.** Definitions of four mutually exclusive cooperation modes computable from four tables common to cooperative platforms (visits, shares, responses, members), together with the log fields each mode depends on (Section 3).
- **A certification procedure.** Four gates applied in sequence, exclusion (G1), minimum support (G2), leakage (G3), and an activity-baseline test (G4), with outcomes reported in a fixed-schema data card (Sections 4–5).
- **An evaluation by defect injection** against a naive performance rule and a stronger practitioner baseline, with gate ablations, defects layered on genuine signal, combined defects, a power analysis, and a stress test of undeclared leakage that identifies where the gates fail (Section 6).
- **A worked application** to eight months of logs from an operating mutual-aid platform (Section 7).

We are explicit about what the evaluation shows. Defect injection establishes that the gates behave as specified against known ground truth; it does not estimate their detection rate on any particular real platform, whose defects may differ in kind or size from those we simulate.

2 Related work

Data validation and documentation. Production ML systems validate incoming data against schemas and statistical expectations (Breck et al., 2019; Schelter et al., 2018), and documentation frameworks standardise what is disclosed about a dataset's provenance and limits (Gebru et al., 2021; Pushkarna et al., 2022). Earlier data-quality work frames quality as fitness for a purpose (Wang & Strong, 1996). These tools check that data are well-formed and documented; they do not test whether a specific supervised label is supported by the data, which is the question here. Our data card borrows the reporting idea and adds a computed verdict per label.

Leakage. Target leakage, a feature that is a proxy for or a consequence of the label, is a long-recognised failure in applied prediction (Kaufman et al., 2012) and a major source of irreproducible results in ML-based science (Kapoor & Narayanan, 2023). It is usually caught with domain knowledge of which features are temporally downstream of the outcome. G3 makes that check explicit for event logs (a declared recording-time test) and adds a univariate tripwire; Section 6.5 measures how much the tripwire catches when the declaration is missing.

Activity confounding. In behavioral and network data, an actor's activity level is a standard alternative explanation for apparent behavioral effects, producing spurious homophily and contagion (Anagnostopoulos et al., 2008; Shalizi & Thomas, 2011). G4 turns this concern into a test: a label is certified only if behavioral features beat an activity-only baseline.

Cascades. Work on online diffusion asks whether and how far content spreads and what shape the cascade takes (Cheng et al., 2014; Goel et al., 2016). That work characterises cascades once they are observed; we address the earlier question of whether role labels within a cascade are valid at all.

3 Cooperation-mode labels

Log model. A platform records, over members M and requests R: visits V, rows $(m, r, t, \text{dwell}, \text{scroll}, s)$ where s is the share through which the visit arrived, if any; shares S, rows (s, m, r, t, p) where p is the parent share when this share was itself reached through a share; responses P, rows (m, r, t); and a member registry sufficient to identify operator, test and insider accounts. Each request has an author $a(r)$. The unit of analysis is an *exposure* (m, r) with $m \neq a(r)$ and m not excluded by G1. Where most traffic is unauthenticated, a session can stand in for a member (Section 7).

Modes. For an exposure (m, r), with priority in the order listed:

1. **chain-origin**: m is a broker (below) and some share of r created by a different member has one of m's shares as its parent;
2. **broker**: m created a share of r through which at least one other member visited r;
3. **direct-help**: m posted a non-deleted response to r;
4. **pass**: m visited r and none of the above holds.

The priority makes the labels mutually exclusive; chain-origin implies broker by construction.

Field dependencies. Two modes depend on fields that operational logs often lack. Broker and chain-origin require each visit to carry the identifier of the share that produced it, including for unauthenticated visitors arriving through external channels; a log that records this only for logged-in visitors silently under-counts both modes. Chain-origin also requires the share table to store the parent share at the moment of re-sharing. A log that records a share's depth without its parent cannot support chain-origin at any sample size. This is a missing field, not a small sample, and it is distinct from the defects the gates test for.

4 The quality gates

Each mode is evaluated independently, as a binary label (this mode versus any other), through four gates in sequence. A label fails at the first gate it does not pass, and that gate is recorded.

G1: exclusion. Remove every exposure involving an account on the exclusion list (operator, developer and test accounts, by role flag, e-mail domain or explicit ID) and every exposure of a request by its own author. Report the number removed.

G2: minimum support. Require at least $n_{\min}$ positive and $n_{\min}$ negative exposures after G1. We use $n_{\min} = 30$ as a conventional default and report the achieved counts so the choice is auditable.

G3: leakage. Drop a feature if (a) its declared recording time is at or after the labelled event, or (b) its univariate AUC against the label, in either direction, exceeds $\tau_{\text{leak}} = 0.95$. Check (a) needs feature metadata; check (b) is a tripwire that needs none but catches only strong leaks. If no feature survives, the label fails at G3.

G4: activity baseline. Fit a logistic model on the surviving features and obtain out-of-fold scores by stratified 5-fold cross-validation. Compare their AUC with the AUC of an activity-only score (the member's number of exposures in the dataset). The label passes only if both (i) the 2.5th percentile of a bootstrap distribution of the AUC difference (model minus baseline) is above zero, and (ii) a permutation test of the model's AUC against label-permuted nulls gives $p < 0.05$. Both resampling procedures hold the out-of-fold scores fixed, which makes them cheap and slightly anti-conservative relative to refitting under each resample.

What passing means. Passing certifies that the label, as built from this log, is not an artifact of the four defects tested. It does not certify that the operational definition captures the construct a researcher cares about, nor that the effect generalises beyond the platform.

5 The data card

Every run emits a JSON report that travels with the labels whether or not they are certified: source files and hashes, excluded account IDs, the exposure unit, counts per mode, and for each mode the gate outcomes (counts, dropped features, model and baseline AUC, bootstrap lower bound, permutation p, verdict). A free-text field records structural notes such as a missing parent-share column. The flag `usable_as_supervised_data` is true only if every requested mode passes, so a failing mode cannot hide behind a passing average. The reference implementation (Appendix B) produces this card from raw platform exports.

Table 1: Certification rates over 500 replications per condition. "True signal" is whether the outcome depends on the behavioral features. Naive AUC: mean (s.d.) of the cross-validated AUC on all features. Last column: gate at which the gates declined, as a share of replications.

Condition	True signal	Med. pos.	Naive AUC	Naive	Practitioner	Gates	Gate that declined
clean	yes	436	0.774 (0.013)	100.0%	100.0%	**100.0%**	—
null	no	276	0.497 (0.025)	0.0%	0.0%	**3.0%**	G4 100%
operator	no	662	0.770 (0.012)	100.0%	100.0%	**2.2%**	G4 100%
test accounts	no	799	0.722 (0.011)	97.6%	97.6%	**3.0%**	G4 100%
leakage	no	278	0.978 (0.004)	100.0%	0.0%	**2.8%**	G4 100%
confound	no	430	0.743 (0.015)	99.8%	99.8%	**0.0%**	G4 100%
scarcity	yes (rare)	9	0.741 (0.110)	69.0%	0.0%	**0.0%**	G2 100%

6 Evaluation by defect injection

No cooperative-platform dataset is annotated with which defects it contains, so we evaluate the gates on synthetic logs whose generative process we control.

6.1 Setup

Generator. Each replication draws 400 members with log-normal activity (exposures per member), giving roughly 2,500 exposures. Each exposure has four behavioral features (standardised dwell, scroll, visit count, and an arrival-through-share indicator). Two base processes are used. In the *signal* base the outcome follows a logistic model of the features with moderate coefficients; in the *null* base it depends on none of them. Defects are injected on top of a base:

- **operator**: 450 extra exposures from three administrative accounts (on the exclusion list) with elevated features and an 85% positive rate; these accounts are also far more active than any user;
- **test accounts**: 150 test or insider accounts (on the exclusion list) with ordinary activity levels but elevated features and a 60% positive rate;
- **leakage**: an extra feature equal to the outcome plus Gaussian noise ($\sigma = 0.35$ unless stated), declared as recorded after the event;
- **confound**: the outcome depends on member activity, and the behavioral features are themselves inflated by activity;
- **scarcity**: the signal base with a much lower intercept, giving a median of 9 positives.

clean is the signal base with no defect and *null* the null base with no defect. In the main experiment, operator, test accounts, leakage and confound are injected on the null base, so any signal a procedure finds is an artifact.

Procedures. The **naive** rule certifies if the 5-fold cross-validated AUC of a logistic model on all available features exceeds 0.70. The **practitioner** baseline adds the checks a careful analyst would apply without an exclusion list or activity baseline: it requires 30 positives and 30 negatives and drops any feature with univariate AUC above 0.95 before applying the naive rule. The **gates** are as in Section 4, with 200 bootstrap and 200 permutation draws.

6.2 Main result

Table 1 shows the pattern the gates were designed to produce. The naive rule accepts every contaminated, leaked or confounded condition at 97.6% or more, because each defect raises cross-validated AUC above 0.70 without any genuine behavioral signal. The practitioner baseline removes the leaked feature through

Table 2: Defects injected on top of genuine behavioral signal. The desired outcome is certification after the defect is handled.

Condition	Naive	Practitioner	Gates
operator + signal	100.0%	100.0%	**100.0%**
leakage + signal	100.0%	100.0%	**100.0%**
confound + signal	100.0%	100.0%	**84.7%**

Table 3: Certification rate with one gate removed at a time (200 replications per condition). A rise against the full pipeline on a defective condition shows the removed gate was needed for that defect. Without G4, surviving features are certified if their cross-validated AUC exceeds 0.70.

Condition	All gates	without G1	without G2	without G3	without G4
clean	100.0%	100.0%	100.0%	100.0%	100.0%
null	2.0%	2.0%	2.0%	2.0%	0.0%
operator	2.0%	1.5%	2.0%	2.0%	0.0%
test accounts	2.0%	100.0%	2.0%	2.0%	0.0%
leakage	3.0%	3.0%	3.0%	100.0%	0.0%
confound	0.0%	0.0%	0.0%	0.0%	99.5%
scarcity	0.0%	0.0%	48.0%	0.0%	0.0%

its univariate check (leak AUC 0.978 on average) and refuses the scarce condition on sample size, but it has no defence against contamination or confounding and accepts them at 97.6%–100.0%. The gates accept defective data in at most 3.0% of replications and accept clean data in all of them. On the null condition the gates' false-certification rate (3.0%) is below the nominal 5% of the permutation test.

The scarcity condition differs from the others: the signal is genuine, but with a median of 9 positives the naive AUC has a standard deviation of 0.110 across replications, four or more times that of every other condition. A naive certification here is a draw from a very wide sampling distribution rather than a stable finding; G2 declines before that estimate is computed.

6.3 Defects layered on genuine signal

A gate that rejects everything would score well in Table 1. To check that the gates do not reject data that is usable once the defect is removed, we inject each defect on the *signal* base (300 replications).

With genuine signal underneath, the gates certify the operator and leakage conditions in 100.0% and 100.0% of replications: G1 and G3 remove the defect and the remaining behavioral signal passes G4. Confounding layered on signal is harder. The gates certify it in 84.7% of replications; in the rest, behavior and activity are too entangled for the bootstrap interval on the AUC difference to clear zero at this sample size. This is the cost of G4 being conservative about activity, and we report it as such.

6.4 Ablation

Each gate is necessary for at least one defect (Table 3). Without G1, test accounts are certified in 100.0% of replications; without G2, scarce data is certified in 48.0%; without G3, leaked data in 100.0%; without G4, confounded data in 99.5%. The high-activity operator case is an exception that matters for interpretation: it is caught by G4 even when G1 is removed (1.5%), because those accounts' activity makes the activity baseline itself predictive. G1 is needed when contaminating accounts look ordinary in activity, which is the more common situation for test and insider accounts.

Table 4: Undeclared leakage on the null base. Certification of this data is always wrong. Leak AUC is the mean univariate AUC of the leaked feature.

Leak noise σ	Leak AUC	Naive	Practitioner	Gates
0.20	1.000	100.0%	0.0%	**4.5%**
0.35	0.979	100.0%	0.0%	**3.0%**
0.50	0.923	100.0%	100.0%	**100.0%**
0.75	0.828	100.0%	100.0%	**100.0%**
1.00	0.761	100.0%	100.0%	**100.0%**
1.50	0.682	4.5%	4.5%	**100.0%**
2.50	0.611	0.0%	0.0%	**96.0%**

Table 5: Two defects at once on the null base (300 replications).

Condition	Naive	Practitioner	Gates
leakage + confound	100.0%	0.0%	**0.0%**
operator + leakage	100.0%	100.0%	**3.3%**

6.5 Undeclared leakage

In the main experiment the leaked feature is declared as recorded after the event, so G3(a) removes it outright. Real pipelines often lack this metadata. We therefore remove the declaration and vary the leak's noise, leaving only the tripwire G3(b) and the downstream G4 (200 replications per level, null base).

This is where the gates fail (Table 4). While the leak is strong enough to exceed the tripwire (univariate AUC of about 0.98 or more, $\sigma \leq 0.35$), G3(b) removes it and the gates decline almost always. Between $\sigma = 0.5$ and 1.0 the leak's univariate AUC falls to 0.76–0.92, below the tripwire, and every procedure, the gates included, certifies the data in all replications. At $\sigma \geq 1.5$ the leak is weak enough that the naive rule's 0.70 threshold rejects it, but the gates still certify in nearly all replications, because G4 tests whether the model beats the activity baseline significantly, not whether the effect is large. For weak, undeclared leaks the gates are therefore more permissive than the naive rule.

Two conclusions follow. First, the tripwire is a backstop for gross leakage only; the reliable defence is the declared recording time of G3(a), which removes the leak at every strength. Pipelines that cannot declare recording times should not treat a passing verdict as evidence against leakage. Second, adding an effect-size floor to G4 would reduce the permissiveness at very weak leaks, but would also lower power for genuine weak effects (Table 6); we leave the default without a floor and report the trade-off.

6.6 Combined defects

When two defects occur together (Table 5), the gates decline in all leakage-plus-confound replications and certify operator-plus-leakage in 3.3%. The practitioner baseline catches the first combination, since removing the leak leaves confounded features below 0.70, but accepts the second in every replication, because removing the leak still leaves the operator contamination.

6.7 Power

The gates' cost on genuine signal depends on effect size and sample size (Table 6). At the main experiment's effect size (1.00) and at 0.75, certification is complete even with 100 members. At a quarter of that effect, certification rises from 25% with about 70 positives to 98% with about 600. A platform with weak behavioral effects therefore needs hundreds of positives per mode before the gates can certify, well above the $n_{\min} = 30$ of G2.

Table 6: Gate certification rate for genuine signal by effect size (coefficient multiplier) and population size; median positives in parentheses (100 replications per cell).

Effect	100 members	200 members	400 members	800 members
0.25	25.0% (74)	50.0% (148)	75.0% (297)	98.0% (595)
0.50	78.0% (82)	98.0% (165)	100.0% (326)	100.0% (660)
0.75	100.0% (95)	100.0% (188)	100.0% (378)	100.0% (753)
1.00	100.0% (107)	100.0% (214)	100.0% (435)	100.0% (865)

7 Application to an operating mutual-aid platform

We applied the reference implementation to eight months of logs (February–September 2026) from an operating mutual-aid request platform. Most third-party visitors on this platform are not logged in, so the exposure unit is a session–request pair. G1 removed operator, insider and test accounts by ID and e-mail domain, any session or IP address ever seen logged in as a member (to remove operator traffic while logged out), 1,164 visits from bot user agents, and 4,897 visits to test-fixture request identifiers. This left 254 session–request exposures.

Table 7: Gate outcomes on eight months of operating-platform logs (session–request exposures, $n = 254$).

Mode	Positives	Negatives	Verdict
pass	250	4	insufficient (G2)
broker	2	252	insufficient (G2)
chain-origin	0	254	not measurable (missing field)
direct-help	2	252	insufficient (G2)

No mode reaches $n_{\min} = 30$ on both sides (Table 7). Two of the four are limited by more than volume. Under the session unit, broker can only be computed from a share-button click, not from a verified share that brought a visitor, and chain-origin cannot be attributed at all because the share table records neither the session nor the IP of the sharer. The data card records these as structural notes, distinct from the sample-size verdicts. The practical consequence differs: for pass and direct-help the remedy is more traffic, while for chain-origin and verified broker the remedy is a change to what is logged.

A naive check on this data would have been misleading in a specific way. The pass mode has 250 positives, enough to look usable, and only the requirement of 30 negatives as well stops it. All four modes stopped at G2 or earlier, so the G4 resampling procedure was not reached, and the verdicts do not depend on the bootstrap settings.

8 Discussion and limitations

What the evaluation shows. The gates behave as specified against defects with known ground truth: they reject each defect we injected, alone or in pairs, while keeping full power for clean signal of moderate size. Each gate is necessary for at least one defect. This establishes internal soundness. It does not estimate detection rates on real platforms, whose defects may take forms or sizes we did not simulate.

The generator is ours. We designed both the defects and the gates, so agreement between them is partly by construction. We have tried to counter this in three ways: a practitioner baseline that a reader might consider sufficient, a stress test aimed at a known blind spot (undeclared leakage) that the gates fail, and defects layered on genuine signal where over-rejection would show. An independent test would require platforms with audited defects, which do not yet exist for this label family.

Limitations. (i) Undeclared leakage below the tripwire passes the gates (Section 6.5). (ii) The thresholds $n_{\min} = 30$ and $\tau_{\text{leak}} = 0.95$ are conventional defaults, not derived; Table 6 shows that 30 positives is far

from sufficient for weak effects. (iii) G4's bootstrap and permutation procedures hold out-of-fold scores fixed, which is cheaper and somewhat anti-conservative compared with refitting under each resample; the null false-certification rates stay below 5% in our runs. (iv) The four modes are tested separately with no correction for multiple labels. (v) The gates address label validity, not label meaning: a certified broker label still encodes one operational definition of brokering, and whether it matches what participants would recognise as their own intent is a construct-validity question we do not address. (vi) The application covers one platform with low traffic, so it demonstrates the data card's diagnostic value rather than a certified label.

9 Conclusion

Cooperation-mode labels built from operational logs are cheap to construct and easy to get wrong in ways a predictive-performance check will not reveal. We defined four such labels over common log tables, a four-gate certification procedure and a data card that reports why a label passed or failed. Under defect injection the gates reject contamination, test accounts, leakage, confounding and scarcity that a naive rule and a practitioner baseline accept, and each gate is necessary for at least one of them. They fail on undeclared leakage weaker than the tripwire, which is why recording-time metadata should be treated as a requirement rather than an option. On a real platform the gates declined all four labels and separated a sample-size shortfall from a missing field. We release the harness and pipeline so that others can test the gates against defects we did not anticipate.

References

Aris Anagnostopoulos, Ravi Kumar, and Mohammad Mahdian. Influence and correlation in social networks. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 7–15, 2008.

Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data validation for machine learning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2019.

Justin Cheng, Lada Adamic, P. Alex Dow, Jon Michael Kleinberg, and Jure Leskovec. Can cascades be predicted? In *Proceedings of the 23rd International Conference on World Wide Web*, pp. 925–936, 2014.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Sharad Goel, Ashton Anderson, Jake Hofman, and Duncan J. Watts. The structural virality of online diffusion. *Management Science*, 62(1):180–196, 2016.

Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.

Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012.

Mahima Pushkarna, Andrew Zaldivar, and Oddur Kjartansson. Data cards: Purposeful and transparent dataset documentation for responsible AI. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pp. 1776–1826, 2022.

Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. Automating large-scale data quality verification. *Proceedings of the VLDB Endowment*, 11(12): 1781–1794, 2018.

Cosma Rohilla Shalizi and Andrew C. Thomas. Homophily and contagion are generically confounded in observational social network studies. *Sociological Methods & Research*, 40(2):211–239, 2011.

Richard Y. Wang and Diane M. Strong. Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4):5–33, 1996.

A Sensitivity of the naive rule to its threshold

Table 8: Naive certification at three thresholds (200 replications per condition at 0.60 and 0.80; 0.70 from Table 1). Gate rates do not depend on the threshold.

Condition	Naive $\tau = 0.60$	Naive $\tau = 0.70$	Naive $\tau = 0.80$	Gates
clean	100.0%	100.0%	2.0%	100.0%
null	0.0%	0.0%	0.0%	3.0%
operator	100.0%	100.0%	0.0%	2.2%
leakage	100.0%	100.0%	100.0%	2.8%
confound	100.0%	99.8%	0.0%	0.0%
scarcity	90.5%	69.0%	31.0%	0.0%

No threshold makes the naive rule reliable (Table 8). At 0.60 it accepts every defect. At 0.80 it rejects most defects but also rejects clean data, whose cross-validated AUC is 0.774 on average, and it still accepts the leakage condition in every replication because the leak pushes AUC near 0.98. The gates do not use this threshold.

B Reference implementation

A public repository accompanies this paper, `https://github.com/kazutakahayashiyorosix/quality-gates-cooperation-labels`, and contains three scripts. `gate_experiment.py` implements the generator, the naive and practitioner baselines, the gates and every experiment in Section 6; `python3 gate_experiment.py all` reproduces Tables 1–6 and 8 with fixed seeds, and `python3 gate_experiment.py extra` adds the test-account condition. `make_tables.py` builds every table and every number quoted in the text from the result files. `pipeline_v2.py` computes the mode labels, the gates and the data card from raw platform exports (members, requests, visits, shares, responses) at member or session level; the platform exports themselves are not released for participant privacy, but the data card produced from them is.